

Global Solution 2026.1

Astra Resilience Copilot

Plataforma inteligente para monitoramento ambiental com dados espaciais, sensores edge, análise de risco e RAG Copilot

Equipe	Equipe Rocket
Integrantes	Jonas Luis da Silva - RM561465 Edson Henrique Felix Batista - RM566321
Tutor(a)	Lucas Gomes Moreira
Coordenador(a)	André Godoi Chiovato
Status para o pódio	QUERO CONCORRER
Repositório GitHub	https://github.com/jonsilva91/Global-Solutions-2026
Vídeo demonstrativo	https://youtu.be/znjxl3e4Tjl

Documento final de entrega - POC acadêmica

Junho de 2026

1. Introdução

O espaço deixou de ser apenas um território científico e passou a representar uma fronteira tecnológica, econômica e estratégica. Satélites apoiam monitoramento climático, agronegócio, segurança, telecomunicações, rastreamento global e geração massiva de dados para tomada de decisão. Nesse contexto, a Global Solution 2026.1 propõe o desenvolvimento de uma POC conectada à nova economia espacial.

O Astra Resilience Copilot foi criado para responder à pergunta: como tecnologias avançadas de Inteligência Artificial, automação e computação podem impulsionar soluções inovadoras para a nova economia espacial? A proposta transforma dados espaciais e sinais de sensores locais em score de risco, evidências rastreáveis, alertas e briefing operacional com IA Generativa/RAG.

Problema escolhido

Eventos ambientais como queimadas, secas, tempestades e enchentes exigem monitoramento rápido, confiável e explicável. Embora plataformas espaciais gerem dados valiosos, esses dados precisam ser integrados a sistemas distribuídos, sensores de campo e mecanismos de apoio à decisão para gerar valor operacional.

Objetivo

- Integrar dados espaciais da NASA FIRMS e NASA EONET com uma simulação de sensor edge/ESP32.
- Calcular um score de risco ambiental explicável, com breakdown por fonte de risco.
- Gerar evidências e proveniência para cada alerta produzido.
- Apresentar os resultados em uma interface React estilo mission console.
- Gerar um briefing operacional com RAG Copilot baseado em contexto local e evidências do risco.

2. Desenvolvimento da solução

Visão geral

A solução é organizada em camadas: fontes de dados espaciais e edge, backend FastAPI, motor de risco, armazenamento local, base de conhecimento RAG e frontend React. A POC foi construída para ser demonstrável em ambiente local, mantendo reprodutibilidade mesmo em cenários sem API key ou com instabilidade em serviços externos.

Astra Resilience Copilot — Diagrama de Arquitetura

Global Solution FIAP 2026.1

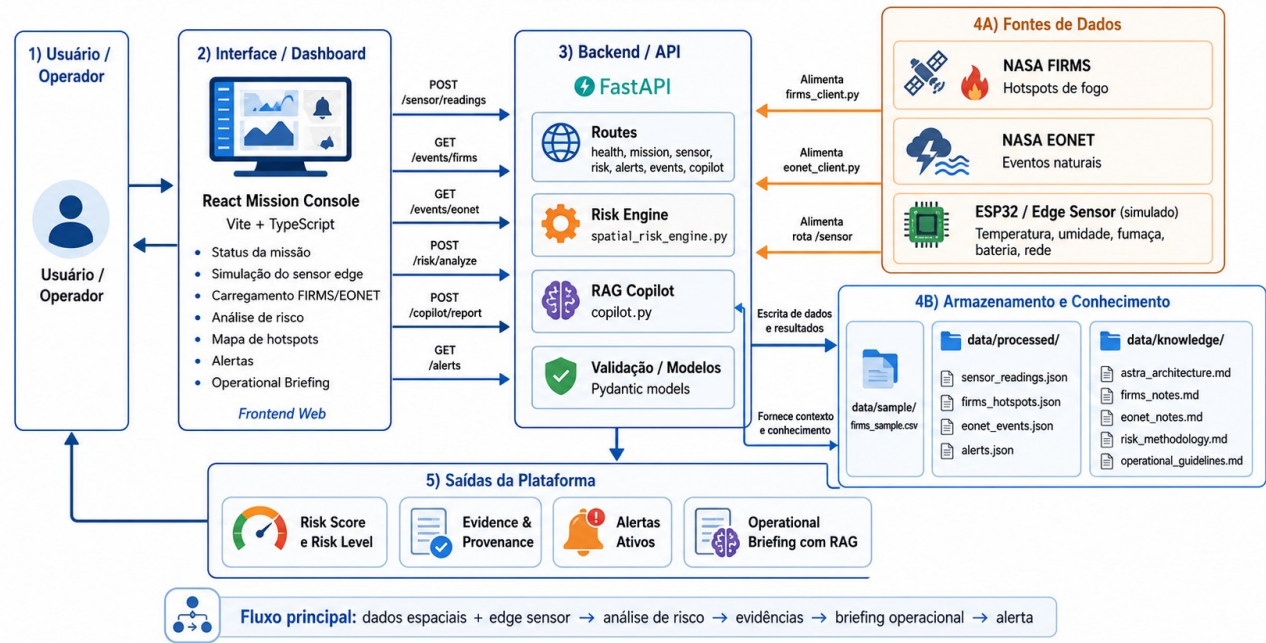


Figura 1 - Diagrama de arquitetura do Astra Resilience Copilot.

Camadas do sistema

Camada	Implementação	Função na POC
Dados espaciais	NASA FIRMS e NASA EONET	Fornecer sinais orbitais e eventos naturais para análise ambiental.
Edge/Fog	Sensor ESP32 simulado	Simular dados locais de temperatura, umidade, fumaça, bateria e rede.
Backend	FastAPI + Pydantic	Expor endpoints, validar dados, processar risco e alertas.
Inteligência	Spatial Risk Engine	Combinar FIRMS, EONET e sensor em score 0-100.
RAG/GenAI	RAG Copilot + Markdown KB	Gerar briefing operacional baseado em evidências e contexto.
Experiência	React Mission Console	Mostrar status, dados, risco, mapa, evidências, alertas e briefing.

Tecnologias utilizadas

Categoria	Tecnologias
Backend e APIs	Python 3.11, FastAPI, Pydantic, Uvicorn, Requests
Frontend	React, TypeScript, Vite, Axios, Recharts, Lucide React, CSS customizado
Dados	NASA FIRMS, NASA EONET, CSV local, JSON local
IA e automação	Motor de risco explicável, RAG Copilot, base Markdown, alertas automáticos
Execução	Windows PowerShell, ambiente virtual Python, npm

Estrutura de pastas

```
Global-Solutions-2026/
├── data/
│   ├── knowledge/      # base local do RAG Copilot
│   ├── processed/      # arquivos JSON gerados em runtime (.gitkeep no repo)
│   └── sample/         # amostra FIRMS para demo reproduzível
├── docs/
│   ├── architecture/   # diagrama de arquitetura
│   ├── screenshots/    # prints da mission console
│   ├── pdf/            # PDF final de entrega
│   └── video/          # link do vídeo demonstrativo
├── frontend/mission-console/ # React + TypeScript + Vite
├── src/
│   ├── backend/        # FastAPI, rotas e modelos
│   ├── ingestion/      # clients FIRMS/EONET
│   ├── intelligence/    # spatial risk engine
│   └── rag/            # copilot.py
├── README.md
├── TESTING.md
├── requirements.txt
└── run.ps1
```

3. Funcionamento implementado

Fluxo operacional

- O usuário acessa a React Mission Console e informa a área de interesse, como Pantanal.
- O painel envia uma leitura simulada de sensor edge para o backend.
- O sistema carrega hotspots do FIRMS e, quando disponível, eventos do EONET.
- O endpoint `/risk/analyze` combina sinais espaciais e locais para gerar score e nível de risco.
- O backend salva evidências, proveniência e alertas em `data/processed/`.
- O RAG Copilot usa o resultado da análise e a base `data/knowledge/` para gerar um briefing operacional.

Endpoints principais

Endpoint	Método	Função
<code>/health</code>	GET	Verifica status da API.
<code>/mission/info</code>	GET	Retorna metadados da missão e componentes do projeto.
<code>/sensor/readings</code>	POST	Recebe leitura simulada do sensor edge.
<code>/sensor/readings/latest</code>	GET	Retorna última leitura de sensor.
<code>/events/firms</code>	GET	Carrega hotspots FIRMS.
<code>/events/eonet</code>	GET	Carrega eventos naturais EONET.
<code>/risk/analyze</code>	POST	Executa análise de risco espacial.
<code>/alerts</code>	GET	Lista alertas ativos.
<code>/copilot/report</code>	POST	Gera briefing operacional com RAG Copilot.

Motor de risco explicável

O motor de risco produz uma pontuação total de 0 a 100. O resultado não é tratado como uma caixa-preta: o dashboard mostra o breakdown por componente, permitindo entender a contribuição de FIRMS, sensor, EONET, fumaça e risco operacional.

```
# Exemplo conceitual do cálculo de risco
```

```
risk_breakdown = {  
  "firms_risk": firms_component,  
  "sensor_risk": sensor_component,  
  "eonet_risk": eonet_component,  
  "smoke_risk": smoke_component,  
  "operational_risk": operational_component,  
}
```

```
total_risk_score = min(100, sum(risk_breakdown.values()))
```

```
risk_level = classify_risk_level(total_risk_score)
```

Sensor edge/ESP32 simulado

A POC simula um dispositivo de borda com temperatura, umidade, umidade do solo, nível de fumaça, bateria e status de rede. Essa decisão permite demonstrar Edge/Fog Computing mesmo sem depender de hardware físico durante a avaliação.

```
{
  "device_id": "esp32_001",
  "temperature": 39,
  "humidity": 18,
  "soil_moisture": 15,
  "smoke_level": 35,
  "battery_level": 35,
  "network_status": "degraded"
}
```

RAG Copilot

O RAG Copilot transforma a análise de risco em briefing operacional. Para reduzir alucinação e manter o relatório contextualizado, a geração é baseada no payload de risco, nas evidências e nos documentos Markdown presentes em data/knowledge/.

```
# Fluxo conceitual do RAG Copilot
context = load_markdown_knowledge_base("data/knowledge")
risk_payload = request.risk_analysis
report = generate_operational_briefing(context, risk_payload)
return report
```

4. Resultados obtidos e evidências visuais

Os prints abaixo demonstram o funcionamento da POC durante a execução local. O fluxo apresentado no vídeo mostra a sequência completa: sensor edge, dados FIRMS, análise de risco, evidências, mapa, alertas e briefing operacional.

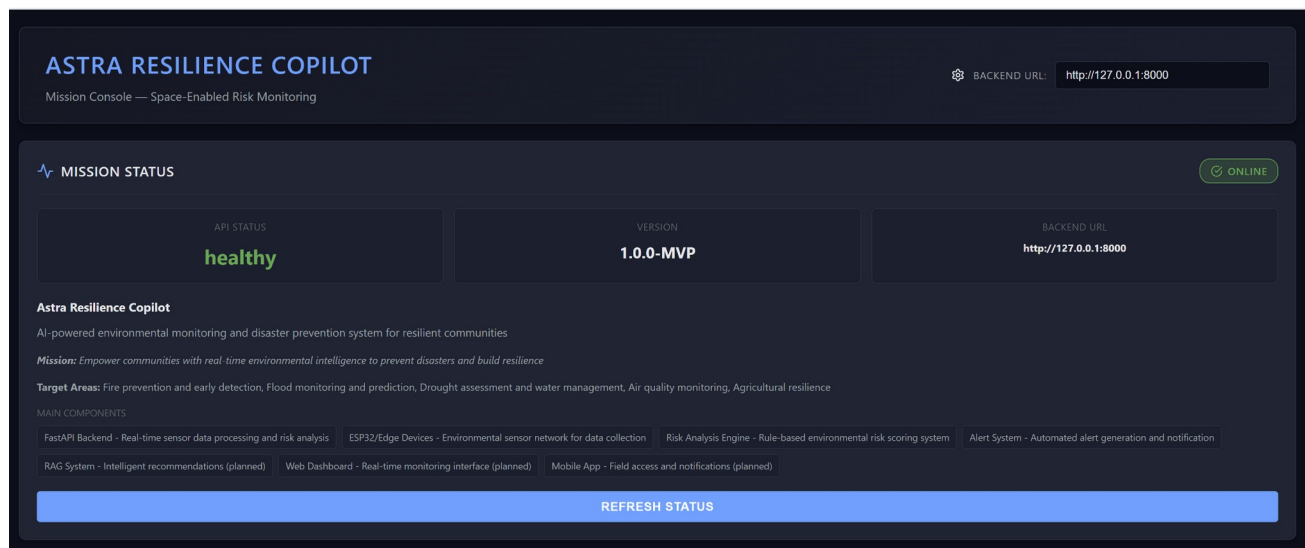


Figura 2 - Mission Status com backend conectado e API saudável.

AREA OF INTEREST

Pantanal

Specify the geographic area for risk analysis and monitoring

EDGE SENSOR SIMULATION

DEVICE ID

esp32_001

TEMPERATURE (°C)

39

HUMIDITY (%)

18

SOIL MOISTURE (%)

15

SMOKE LEVEL (PPM)

35

BATTERY LEVEL (%)

35

NETWORK STATUS

Degraded

SUBMIT SENSOR READING

Sensor reading stored successfully

SPACE DATA INGESTION

FIRMS HOTSPOTS

15

EONET EVENTS

0

LOAD FIRMS HOTSPOTS

LOAD EONET EVENTS

Loaded 15 FIRMS hotspots successfully!

[object Object]. This is expected if NASA EONET API is unavailable. You can continue with FIRMS data.

FIRMS: NASA Fire Information for Resource Management System - Active fire hotspots from MODIS/VIIRS satellites

EONET: Earth Observatory Natural Event Tracker - Global natural events including wildfires, storms, and volcanic activity

Figura 3 - Sensor edge simulado e ingestão de hotspots FIRMS.

SPACE-ENABLED RISK ANALYSIS

RUN SPACE-ENABLED RISK ANALYSIS

Risk analysis completed successfully!

RISK SCORE

71.0

TOTAL RISK SCORE

71.0

RISK LEVEL

HIGH

RECOMMENDED ACTION

HIGH RISK: Prepare for potential evacuation. Active fires detected nearby - monitor fire spread closely. Elevated smoke levels - ensure air filtration and limit outdoor exposure.

AREA OF INTEREST

Pantanal

Figura 4 - Análise de risco espacial com score 71.0 e nível HIGH.



Figura 5 - Breakdown do risco por FIRMS, sensor, EONET, fumaça e risco operacional.

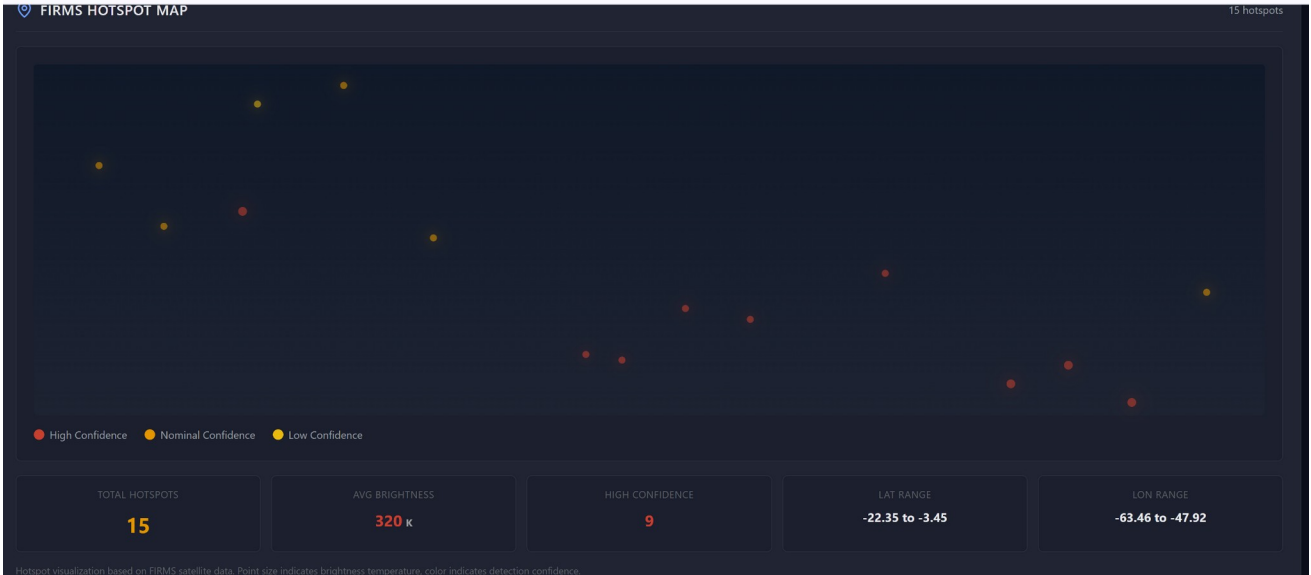


Figura 6 - Mapa de hotspots FIRMS com distribuição geográfica e confiança das detecções.

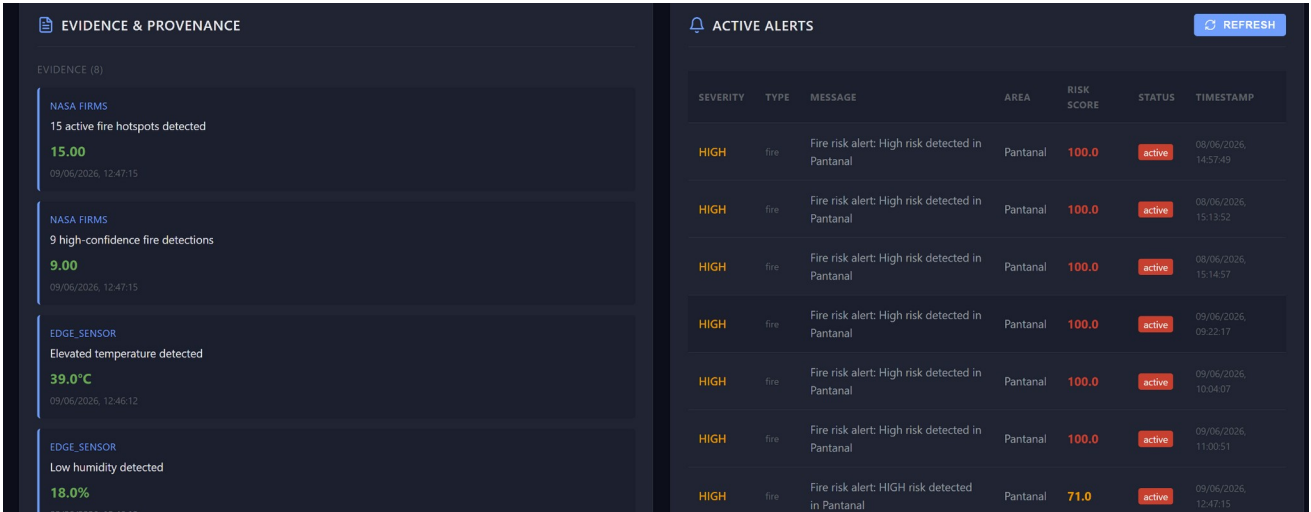


Figura 7 - Evidências, proveniência e alertas ativos gerados pela análise.

5. Resultados esperados

Como prova de conceito, a solução demonstra que dados derivados do espaço podem ser integrados a sinais edge e a uma camada de IA para produzir resultados úteis, explicáveis e acionáveis.

Resultado	Descrição
Monitoramento inteligente	Integração de hotspots, eventos naturais e sensor local em uma visão operacional.
Score de risco	Pontuação de 0 a 100 com classificação LOW, MODERATE, HIGH ou CRITICAL.
Explicabilidade	Breakdown de risco e cards de evidência para justificar o resultado.
Proveniência	Indicação dos arquivos e fontes que alimentam a análise.
Alertas	Geração automática de alertas quando o risco é elevado.
Briefing operacional	Resumo com recomendações baseado no RAG

	Copilot.
--	----------

6. Aderência aos requisitos da Global Solution

Requisito FIAP	Como o projeto atende
IA Generativa, automação inteligente ou ML	RAG Copilot gera briefing operacional; motor de risco automatiza análise e decisão inicial.
Análise de dados, APIs ou dashboards inteligentes	FastAPI, React Mission Console, ingestão FIRMS/EONET e visualizações de risco.
Sensores, ESP32, Edge/Fog ou automação	Simulação de sensor ESP32/edge com temperatura, umidade, fumaça, bateria e rede.
Dados espaciais/economia espacial	Uso de NASA FIRMS e NASA EONET para dados orbitais/eventos naturais.
Planejamento e documentação	README, TESTING, docs, screenshots, diagrama, vídeo e PDF final.
Comunicação visual clara	Dashboard estilo mission console, gráficos, mapa e evidências.
Vídeo demonstrativo	Vídeo publicado no YouTube como não listado e linkado neste PDF.
Código em formato texto	Trechos relevantes apresentados em texto; código completo no repositório.

7. Como executar

Backend

```
git clone https://github.com/jonsilva91/Global-Solutions-2026.git
cd "Global Solutions 2026"
py -3.11 -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m uvicorn src.backend.main:app --host 127.0.0.1 --port 8002
```

Frontend

```
cd frontend/mission-console
npm install
npm run dev -- --host 127.0.0.1 --port 5173
```

No campo Backend URL da interface React, usar: <http://127.0.0.1:8002>

Sequência de demonstração

- Abrir a mission console e confirmar API saudável.
- Enviar uma leitura de sensor de alto risco.
- Carregar hotspots FIRMS.
- Executar a análise de risco espacial.
- Mostrar score, nível de risco, breakdown e mapa.

- Exibir evidências, proveniência e alertas.
- Gerar o briefing operacional com o RAG Copilot.

8. Limitações e próximos passos

Limitações

- O sensor ESP32 foi simulado, não conectado a hardware físico nesta versão da POC.
- O FIRMS utiliza amostra local para garantir demo reproduzível; integração completa com API real pode exigir chave e configuração adicional.
- O EONET depende de disponibilidade de internet/API externa.
- O motor de risco é baseado em regras explicáveis, não em modelo geoespacial treinado com séries históricas extensas.

Evoluções futuras

- Conectar ESP32 físico com sensores reais e envio via Wi-Fi/MQTT.
- Integrar dados Copernicus, HLS ou outras fontes orbitais com maior granularidade.
- Adicionar modelos de Machine Learning para previsão temporal e espacial de risco.
- Criar versão mobile para operação em campo.
- Adicionar autenticação, persistência em banco de dados e deploy em nuvem.

9. Conclusões

O Astra Resilience Copilot demonstra como tecnologias de IA, automação, APIs, sensores edge e dados espaciais podem impulsionar soluções inovadoras para a nova economia espacial. A POC não apenas exibe dados, mas transforma sinais orbitais e locais em análise de risco, evidências, proveniência, alertas e briefing operacional.

A solução atende ao objetivo da Global Solution ao conectar conhecimentos das Fases 3 e 4 em uma aplicação distribuída, com backend, frontend, análise de dados, IA Generativa/RAG e simulação edge. O resultado é uma plataforma demonstrável para prevenção de desastres e resiliência ambiental, com potencial de evolução para uso real em monitoramento remoto.

10. Links finais

Repositório GitHub	https://github.com/jonsilva91/Global-Solutions-2026
Vídeo demonstrativo - YouTube não listado	https://youtu.be/znjxl3e4TjI
Status para pódio	QUERO CONCORRER

Fim do documento